

NS-SBM

A Course in Incompressible Navier–Stokes on an
Immersed-Boundary GPU Solver
Monolithic VMS · Pressure Projection · Nitsche · SBM

DiffSim project

Abstract

This course teaches the simulation of *incompressible Navier–Stokes flow* on DiffSim’s GPU-native, equal-order (P_1/P_1) residual-based-VMS solver. It is built around two production engines: the **monolithic VMS saddle solver** — the correctness *oracle* — and the **pressure-projection (Helmholtz–Leray) solver** — the *scalable* engine whose symmetric-positive-definite pressure step is the path to hundred-million-DOF simulations. Along the way it teaches weak imposition of Dirichlet conditions (**Nitsche’s method**) and the **Shifted Boundary Method** (SBM), which lets an immersed body sit *off* the mesh grid. Every concept pairs a written derivation — read from the code, with `file:line` citations — with a runnable module; you reproduce the drag and centerline numbers here, then explore the knobs on your own. Every “expected result” number was produced by running the module on the GPU box, not invented.

Contents

1	Using this course	3
2	Governing equations and the two engines	4
2.1	Strong form	4
2.1.1	Boundary conditions	4
2.1.2	Nondimensionalization	4
2.2	Equal-order P_1/P_1 and why we stabilize	4
2.3	The two engines	5
2.4	The Shifted Boundary Method in brief	5
3	The monolithic VMS engine	6
3.1	Galerkin weak form	6
3.2	Residual-based VMS: the fine scale $u' = -\tau_M R_M$	6
3.3	The stabilization parameters	6
3.4	The saddle system $\begin{bmatrix} F & G \\ D & C \end{bmatrix}$	7
3.5	Time integration and convection linearization	7
3.6	Code walkthrough	7
3.7	Verification: the lid-driven cavity vs Ghia	7
4	The pressure-projection engine	9
4.1	The Helmholtz–Leray split	9
4.2	The VMS-consistent PPE (<code>consistent_projection</code>)	9
4.3	Fine scale threaded through all three sub-solves	10
4.4	The composed SBM engine	10
4.5	Verification: the same cavity, the same answer	10
5	Weak Dirichlet imposition — Nitsche’s method	12
5.1	Why impose $u = g$ weakly?	12
5.2	The Nitsche boundary terms	12
5.3	Code — <code>sbm_vector_dirichlet</code>	13
5.4	Verification: flow past a square ($d = 0$)	13
6	The Shifted Boundary Method	14
6.1	Surrogate vs true boundary	14
6.2	The Taylor shift $S = N + \nabla N \cdot d$	14
6.3	Surrogate normal, distance, area correction	14
6.4	Force / drag on the immersed body	15

6.5	Verification: a genuine sub-cell shift	15
7	Three dimensions — the SBM sphere	17
7.1	Geometry	17
7.2	The honest 3-D verdict	17
7.3	Solver path	18
7.4	Verification	18
8	Toward 100M DOF (planned)	19
9	Multi-GPU (planned)	21
10	Differentiable structure (planned)	23

Chapter 1

Using this course

Read a concept below, then open its module folder (`tutorials/ns-sbm-course/<NN_name>/`), run `run.py`, and compare your printed self-check table and figures with the ones in this document. The modules call the same production steppers (`diffsim.steppers.*`) and SBM code (`diffsim.sbm.*`) the research code uses — there are no toy re-implementations.

The spine of the course is a single comparison, repeated at rising complexity: *does the scalable pressure-projection split reproduce the same-mesh monolithic oracle?* Chapters 1–2 establish the two engines on a closed cavity; Chapter 3 adds an immersed body with weak (Nitsche) no-slip at $d = 0$; Chapter 4 turns on a genuine sub-cell SBM shift ($d \neq 0$); Chapter 5 goes to 3-D. Chapters 6–8 are the forward-looking placeholders (scaling to 100M DOF, multi-GPU, and differentiable structure).

Boxes flag the two things you act on:

Key idea. The load-bearing concept.

Run it. The command, and what to compare.

Questions marked **Q** are for you to explore — they have no answers printed here on purpose.

Chapter 2

Governing equations and the two engines

Module: 00_setup_and_smoke_test/. Read this before running anything — it sets the notation and the two engines the whole course compares.

2.1 Strong form

On a domain Ω with density 1 and kinematic viscosity ν , the incompressible Navier–Stokes equations for velocity u and pressure p are (`ns_bricks.py:11--14`):

$$\begin{aligned}\partial_t u + (u \cdot \nabla)u &= -\nabla p + \nu \nabla \cdot (\nabla u) + f && \text{(momentum),} \\ \nabla \cdot u &= 0 && \text{(incompressibility).}\end{aligned}\tag{2.1}$$

2.1.1 Boundary conditions

- **Dirichlet** (inflow, walls, moving lid, no-slip body): $u = g$ on Γ_D . Imposed *strongly* (row replacement) or *weakly* (Nitsche, Ch. 5; SBM, Ch. 6).
- **Outflow / “do-nothing”**: the natural boundary condition of the weak form. Integrating the viscous and pressure terms by parts drops their surface terms at the outflow, leaving $\nu \partial_n u - p n = 0$ (a traction-free condition) (`ns_bricks.py:16--17`).

2.1.2 Nondimensionalization

The examples fix a reference speed U and a body length scale D and set the viscosity from the Reynolds number,

$$\nu = \frac{U D}{\text{Re}}.\tag{2.3}$$

In the channel fixtures $U = U_{\text{in}} = 1$ and $D = 2 \times \text{half}$ (the obstacle side length); the cavity uses $D = L = 1$. Drag is reported as $C_d = F_x/q_{\text{ref}}$ with $q_{\text{ref}} = \frac{1}{2}U^2 D$ in 2-D and $\frac{1}{2}U^2 \pi R^2$ for the sphere.

2.2 Equal-order P_1/P_1 and why we stabilize

DiffSim uses **equal-order** (P_1/P_1 , and P_2/P_2) collocated velocity–pressure elements: `ndof = dim + 1` per node, node-major $(u_1, \dots, u_{\text{dim}}, p)$. Equal-order pairs violate the inf–sup (LBB) condition,

so the pure Galerkin saddle system is singular / oscillatory in pressure. **Both** engines therefore add residual-based VMS/SUPG–PSPG stabilization on the momentum strong residual (`ns_bricks.py: 23--27`). The PSPG term is the one that makes equal-order work; without it the pressure–pressure block is empty and the saddle is singular. The full derivation is Chapter 3.

Key idea. Equal-order collocated elements are cheap and simple but LBB-unstable. Residual-based VMS is not optional polish — it is what makes the pressure well-posed at all. The same τ_M, τ_C appear in *both* engines.

2.3 The two engines

	Monolithic (oracle)	Projection (scalable)
Class	<code>LinearizedMonolithicStepper</code>	<code>LerayProjectionStepper</code>
Solve	one coupled (u, p) saddle	predictor $\rightarrow$ SPD PPE $\rightarrow$ correct
Operator	indefinite $\begin{bmatrix} F & G \\ D & C \end{bmatrix}$	one nonsym. + two SPD sub-solves
Scales via	direct / block-precond. FGMRES	SPD PPE $\rightarrow$ AMG/CG
Source	<code>steppers/linearized.py</code>	<code>steppers/leray.py</code>

The **monolithic** solver is the *correctness oracle*: robust, and its steady state is the bar every projection run is measured against. The **projection** solver is the *scalability lever*: its pressure step is a symmetric positive-definite (SPD) Poisson problem that algebraic multigrid handles at scales where the indefinite saddle factorization runs out of memory. For 3-D drag on today’s feasible meshes the monolithic path is the working one (Ch. 7); the projection engine is the path to the 100M-DOF hero (Ch. 8).

Run it. In `00_setup_and_smoke_test/`, run `python doctor.py` (environment gate) then `python run.py` (a tiny lid-driven cavity through *both* engines). The smoke passes when both engines march to a finite, bounded state and the projection tracks the monolithic; compare with `EXPECTED.md`.

2.4 The Shifted Boundary Method in brief

The SBM lets an immersed body sit *off* the mesh grid. Instead of body-fitting, we keep a grid-aligned octree and pick a **surrogate boundary** $\tilde{\Gamma}$ (whole element faces that hug the true boundary Γ). A **Taylor shift** transfers the boundary condition from $\tilde{\Gamma}$ to Γ using a distance vector d (surrogate Gauss point $\rightarrow$ closest point on Γ). When the body *is* grid-aligned, $d = 0$ and SBM degenerates to ordinary Nitsche — which is why the modules build up from $d = 0$ (Ch. 5) to $d \neq 0$ (Ch. 6, Ch. 7).

Q2.4.1 W

hat does q_{ref} become for a 3-D sphere of radius R , and why is it πR^2 rather than $\pi R^2/2$?

Q2.4.2 O

n an equal-order pair, is the pressure defined uniquely for a closed cavity? What does the code pin, and where (see Ch. 4)?

Chapter 3

The monolithic VMS engine

Module: 01_monolithic_vms/. The oracle. One coupled saddle solve per step; the lid-driven cavity vs Ghia.

3.1 Galerkin weak form

Test momentum with w and continuity with q . Integrate the viscous and pressure terms by parts and drop the do-nothing surface terms (`ns_bricks.py:16--21`):

$$\int_{\Omega} w \cdot \partial_t u + \int_{\Omega} w \cdot (u \cdot \nabla) u + \nu \int_{\Omega} \nabla w : \nabla u - \int_{\Omega} (\nabla \cdot w) p = \int_{\Omega} w \cdot f, \quad (3.1)$$

$$\int_{\Omega} q (\nabla \cdot u) = 0. \quad (3.2)$$

3.2 Residual-based VMS: the fine scale $u' = -\tau_M R_M$

VMS splits $u = u_h + u'$ into a resolved (coarse) scale u_h and an unresolved (fine) scale u' , modeled algebraically as $u' = -\tau_M R_M(u_h, p_h)$, where R_M is the momentum strong residual. Substituting u' back into the weak form and integrating the fine-scale terms produces three stabilization contributions (`ns_bricks.py:23--27, 44--51`):

- **SUPG** (streamline-upwind on the momentum test): $\tau_M (a \cdot \nabla w) \cdot R_M$;
- **PSPG** (pressure-stabilizing, on the continuity test): $\tau_M (\nabla q) \cdot R_M$ — *this is the term that makes equal-order work*;
- **grad-div / LSIC**: $\tau_C (\nabla \cdot w) (\nabla \cdot u)$.

3.3 The stabilization parameters

The monolithic stepper uses the **metric-tensor** τ form (`physics/vms.py:5--12, 38--45`). On Diff-Sim's axis-aligned octree cubes the element metric is diagonal, $G = (2/h)^2 I$, so

$$\tau_M = \left[\left(\frac{2b_0}{\Delta t} \right)^2 + \frac{4|u|^2}{h^2} + C_I \nu^2 \dim \left(\frac{2}{h} \right)^4 \right]^{-1/2}, \quad C_I = 36, \quad \tau_C = \frac{1}{\tau_M \dim(2/h)^2}. \quad (3.3)$$

The transient term $(2b_0/\Delta t)^2$ is present when `timestab=True` (dropped for steady/order studies). The device functions are `tau_m_metric / tau_c_metric` (`physics/vms.py:60--74`).

Key idea. The projection engine instead uses the *h-based* τ form, whose defaults reproduce the metric form on cubes *by construction* (`physics/vms.py:13--19`). So the two engines are comparable term-for-term on these meshes — which is exactly what makes the faithfulness comparison meaningful.

3.4 The saddle system $\begin{bmatrix} F & G \\ D & C \end{bmatrix}$

After linearization, each node block is the full $(\dim + 1) \times (\dim + 1)$ coupling written by the element kernel `lin_ns_Ae` (`ns_bricks.py:341--447`):

- F (velocity–velocity): mass $\sigma N_a N_b$ + convection $N_a(a \cdot \nabla N_b + s(\nabla \cdot a)N_b)$ + viscous $\nu \nabla N_a : \nabla N_b$ + SUPG $\tau_M(a \cdot \nabla N_a)R_{M,b}$ + grad-div $\tau_C \partial_i N_a \partial_j N_b$.
- G (velocity–pressure): $-\partial_i N_a N_b$ plus the SUPG pressure coupling $\tau_M(a \cdot \nabla N_a) \partial_i N_b$.
- D (continuity–velocity): $N_a \partial_i N_b$ plus the PSPG velocity coupling $\tau_M \partial_i N_a R_{M,b}$.
- C (the *PSPG C-block*): $\tau_M \nabla N_a \cdot \nabla N_b$. This nonzero pressure–pressure block is exactly what stabilizes the equal-order pair (`ns_bricks.py:442--444`).

3.5 Time integration and convection linearization

The transient term uses backward differencing (BDF1 bootstrap $\rightarrow$ BDF2). Per step $\sigma = b_0/\Delta t$ enters the mass term and τ_M ; the history $-(b_1 u^n + b_2 u^{n-1})/\Delta t$ becomes part of the linearized strong residual (`linearized.py:181--226`). The nonlinear convection $(u \cdot \nabla)u$ is handled by an **advecting field** a precomputed at Gauss points — a fine-scale-corrected BDF *extrapolation* (Picard/Oseen, default) or Newton (adds the cross-term block). The convection uses the **skew-symmetric** form $a \cdot \nabla u + s(\nabla \cdot a)u$ with $s = \frac{1}{2}$ (energy-stable).

3.6 Code walkthrough

`assemble_linear_ns(dm, aq, div_aq, fq, nu, sigma, ...)` (`ns_bricks.py:507--562`) builds the constrained node-major saddle matrix A and RHS b ; it launches `lin_ns_Ae` (matrix) and `lin_ns_be` (RHS) per element bin, then constrains with $T_{\text{vec}} = T \otimes I_{\dim+1}$. `LinearizedMonolithicStepper.step()` (`linearized.py:181--245`) orchestrates one step: BDF coefficients, build a , assemble, apply strong velocity Dirichlet rows + a single pressure pin, solve, rotate history.

3.7 Verification: the lid-driven cavity vs Ghia

The module marches the *same mesh* with *both* engines and compares the $x = 0.5$ centerline $u(y)$ against Ghia, Ghia & Shin (1982), Table I. On a level-4 (16×16) mesh at $\text{Re} = 100$:

station y	projection u	monolithic u	Ghia u
0.9766	+-	+-	+-

$$\max |\text{proj} - \text{mono}| = --, \quad \max |\text{proj} - \text{Ghia}| = --, \quad \max |\text{mono} - \text{Ghia}| = --.$$

The coarse level-4 mesh cannot resolve Ghia's 129^2 table to the third digit — that is expected. The decisive checks are (a) the projection tracks the monolithic on the *same* mesh and (b) both track Ghia within the coarse-mesh tolerance.

Key idea. The pointwise $\|\nabla \cdot u\|$ of the equal-order VMS scheme is *not* driven to zero (— here for the monolithic). The scheme controls the *weak* divergence; a finite, bounded $\|\nabla \cdot u\|$ is correct, and it is the wrong steady-state gate.

Run it. `python run.py` in `01_monolithic_vms/` marches both engines and prints the centerline table. `--level 5` tightens toward Ghia (slower); `--Re 400` needs more steps.

Q3.7.1 T

he viscous term $-\nu \Delta u_h$ in the linearized residual is identically zero at P_1 but required at P_2 . Why? (Hint: second derivatives of $Q1$ shape functions.)

Q3.7.2 F

orce `--solver cudss` on this small cavity. Does it help or hurt? Relate your answer to whether the saddle is definite.

Chapter 4

The pressure-projection engine

Module: 02_pressure_projection/. The scalable engine. Predictor $\rightarrow$ SPD pressure-Poisson $\rightarrow$ velocity correction; the same lid-driven cavity, the same answer; and why this is the 100M-DOF path.

4.1 The Helmholtz–Leray split

The projection engine implements Algorithm 1 of the Helmholtz–Leray VMS draft (`leray.py:1--23`). Per step, with p^* the extrapolated pressure (order 1: $p^* = \hat{p}^n$):

1. **Momentum predictor** (nonlinear, Picard): solve the monolithic block with the pressure DOFs pinned to p^* ’s nodal values, so the momentum rows see ∇p^* through the coupling (`leray.py:6--13, 719--830`).
2. **PPE** — the SPD pressure-Poisson for the increment $\phi = \hat{p} - p^*$:

$$(\nabla\phi, \nabla q) = \sigma (\hat{u} - \tau_M r_m(\hat{u}, p^*), \nabla q)$$

with the h-based τ_M (`leray.py:14--19, 948--1106`).

3. **Velocity correction** (fine scale retained): $u^{n+1} = \hat{u} - \frac{1}{\sigma}(\nabla\hat{p} - \nabla p^*)$.
4. $p^* \leftarrow \hat{p}$ (first-order incremental).

Key idea — why projection scales. The pressure step is a **symmetric positive-definite** Poisson problem. Unlike the indefinite saddle, it is the natural target for algebraic multigrid + CG — the mandated path to the 100M-DOF hero (`leray_sbm.py:19--23`). This is the whole reason projection exists alongside the monolithic oracle.

4.2 The VMS-consistent PPE (`consistent_projection`)

Naïvely splitting the projection *breaks faithfulness*: the split’s steady state is generally *not* the monolithic steady state. The `consistent_projection` mode (the 2026-07-23 fix, `leray.py:48--72`) is the single switch that turns on the coherent VMS-stabilized Helmholtz–Leray set so the monolithic steady state becomes a **fixed point** of the split. It bundles six items:

#1 PSPG-consistent PPE operator the coarse divergence is kept *collocated* with the test q (NOT integrated by parts), so the split’s discrete incompressibility matches the monolithic PSPG continuity row term-for-term (`leray.py:1007--1034`). The full-flux by-parts path fabricates a spurious boundary term $\sigma(u_h \cdot n, q)_\Gamma$ at an open outflow.

#2 fine scale $u' = -\tau_M r_m$ in BOTH the PPE source AND the L2 correction with the consistent mass $D = G^\top$ (`leray.py:53--61`).

#3 disjoint outflow BCs (Baskar’s rule) velocity is left free at the outflow (natural do-nothing $\partial_n u = 0$), while the pressure *correction* $p' = 0$ is imposed as a Dirichlet condition on the outflow nodes. Authoritative: $\nabla u \cdot n = 0$ in the predictor *only*, $p' = 0$ in the PPE *only* (`leray.py:62--69`).

#4 rotational-incremental pressure update (Timmermans 1996) $p = p^* + \phi - \nu \nabla \cdot \hat{u}$, valid for constant ν (`leray.py:1114--1168`).

#5/#6 a P_1 boundary-vorticity outflow term and backflow (directional do-nothing) stabilization, both on the outflow.

Two secular-drift cures ride with the mode: the rotational correction q is pinned to zero on the outflow rows and, for immersed walls, on the SBM/wall nodes — the latter is **the drag fix** for weak-Nitsche bodies (Ch. 5).

4.3 Fine scale threaded through all three sub-solves

The fine-scale velocity $-\tau_M r_m$ is computed once in the PPE pass and *stashed* (`fs_vel`, `leray.py:973`) so the same u' enters the PPE source *and* the velocity correction — the consistency that makes the correction a true discrete projection.

4.4 The composed SBM engine

`LeraySBMStepper` (`leray_sbm.py:81--160`) *composes* the base projection stepper with an immersed-geometry oracle (it does not fork it). It runs the surrogate pipeline once, builds the shifted-Nitsche face block once (`sbm_vector_dirichlet`), and injects it into the predictor via the `extra_block/sbm_nodes` hook. The box inflow/walls are strong; the immersed body is weak (SBM). This is the engine of Ch. 6–7.

4.5 Verification: the same cavity, the same answer

The module re-runs the Chapter 3 lid-driven cavity through the projection engine on the same level-4 mesh. The projection tracks the monolithic to within the coarse-mesh band:

$$\max |\text{proj} - \text{mono}| = --, \quad \max |\text{proj} - \text{Ghia}| = --.$$

The pointwise projection $\|\nabla \cdot u\| = --$ is finite and bounded (not zero) — the same weak-divergence story as Ch. 3. The right divergence gate is the **PPE-space solenoidality identity** $\|\sigma B^\top \hat{u} - K_p \phi\| \approx 0$, which is machine-zero at the fixed point.

Run it. `python run.py` in `02_pressure_projection/` reproduces the cavity through the projection engine and checks it against the monolithic. Try `--no-consistent` to drop `consistent_projection` and watch faithfulness degrade.

Q4.5.1 E

nclosed flow pins one pressure DOF (free-node 0); external flow imposes $p' = 0$ on the outflow. Why can't external flow use the node-0 pin, and why can't enclosed flow use an outflow Dirichlet?

Q4.5.2 T

he base “strong” split on an *open* outflow (without the consistent machinery) is pressure-unstable and can blow up. Which of the six `consistent_projection` items most directly cures that, and why?

Chapter 5

Weak Dirichlet imposition — Nitsche's method

Module: 03_weak_dirichlet_nitsche/. Flow past a square; strong vs weak no-slip; the SBM shift is OFF ($d = 0$), so this is standard Nitsche.

5.1 Why impose $u = g$ weakly?

The strong way to set $u = g$ on Γ_D is to overwrite matrix rows. That works for grid-aligned Dirichlet boundaries, but it does *not* generalize to a boundary that cuts through elements — which is exactly what an immersed body does. Nitsche's method imposes the Dirichlet condition **weakly**, through consistent boundary integrals, and is the foundation the Shifted Boundary Method (Ch. 6) builds on.

5.2 The Nitsche boundary terms

For the component-diagonal (Laplacian) viscous form DiffSim uses — each velocity component gets the scalar Poisson-Nitsche terms with $\kappa \rightarrow \nu$ (`sbm/vector.py:1--14`) — the added boundary terms on Γ_D are

$$\underbrace{-\nu \langle \partial_n u, w \rangle_{\Gamma_D}}_{\text{consistency}} \underbrace{-\nu \langle u - g, \partial_n w \rangle_{\Gamma_D}}_{\text{adjoint-consistency}} + \underbrace{\frac{\alpha\nu}{h} \langle u - g, w \rangle_{\Gamma_D}}_{\text{penalty}}. \quad (5.1)$$

- **Consistency** recovers the exact solution — the true u satisfies the weak form with these terms.
- **Adjoint-consistency** symmetrizes the bilinear form (optimal L^2 convergence) and vanishes when $u = g$.
- **Penalty** provides coercivity; the scale $\alpha\nu/h$ balances the consistency term. Too small $\rightarrow$ the condition leaks; too large $\rightarrow$ ill-conditioning. The examples use $\alpha = 10$.

Key idea. The penalty is the *only* term with a free dimensionless constant α ; setting $\alpha = 0$ removes the penalty and the obstacle stops being felt (an anti-vacuity check). The consistency and adjoint-consistency terms carry no tunable constant — they are exact.

5.3 Code — `sbm_vector_dirichlet`

`sbm_vector_dirichlet(dm, sf, geo, g_fn, nu, ndof, alpha=10.0, ...)` (`sbm/vector.py:23--82`) assembles the vector Nitsche block over full node-major DOFs. It *reuses the verified scalar Poisson-Nitsche face kernels* and places each scalar block at node-major position `node*ndof + c` for component $c < \text{dim}$ — no new device code, the scalar patch-exactness carries over verbatim. The boundary data is evaluated at the *mapped* (true-boundary) point $g(\mathbf{x}_q + d)$; this is where the SBM shift d enters (Ch. 6). At $d = 0$ the mapped point is the surrogate point and this is ordinary Nitsche.

5.4 Verification: flow past a square ($d = 0$)

A cell-aligned square ($\text{half} = 0.125 = k/2^{\text{level}}$) carved from a unit-box channel with `lam=0.0`, so the surrogate faces coincide with the true box: $d_{\max} = 0$, `corr = 1` — standard Nitsche, no shift. Strong inflow + lateral walls; free outflow. Two no-slip modes: *strong* (obstacle nodes join the strong Dirichlet set) and *weak* (Nitsche block; obstacle nodes stay free). The projection runs `consistent_projection`; the weak path additionally uses the rotational *wall pin* (`rot_pin_wall=True`) — the drag fix for weak bodies.

At level 4, $\text{Re} = 40$, weak Nitsche:

	projection	monolithic	rel-diff
C_d	+-	+-	-
mean $ u $	-	-	-

The weak-Nitsche projection matches the same-mesh weak-Nitsche monolithic in both drag and velocity — the Nitsche layer preserves faithfulness on the working projection.

Note. The base single-pass “strong” split on an *open* outflow (without the consistent-projection machinery) is pressure-unstable and can blow up — a known finding. The validated, robust path is the weak `consistent_projection` one shown here.

Run it. `python run.py` in `03_weak_dirichlet_nitsche/` runs the weak path by default. `--mode strong` switches to row-replacement no-slip; `--alpha 0` removes the penalty (the obstacle vanishes — watch $C_d \rightarrow 0$).

Q5.4.1 S

weep $\alpha \in \{1, 5, 10, 50, 100\}$ at fixed mesh. Plot C_d vs α . Where does the condition leak (too small) and where does conditioning degrade (too large)?

Q5.4.2 T

he strong and weak modes give slightly different C_d . Which is “more correct” on a coarse mesh, and how would you decide? (Hint: refine.)

Chapter 6

The Shifted Boundary Method

Module: 04_shifted_boundary/. Flow past a body whose boundary is OFF the grid; the Taylor shift $d \neq 0$ does real work; surrogate-traction drag.

6.1 Surrogate vs true boundary

The true immersed boundary Γ (a level set of an oracle — a `Box` or `Sphere`) generally cuts through octree cells. SBM does *not* body-fit. Instead it classifies cells and keeps a **surrogate boundary** $\tilde{\Gamma}$ made of *whole* element faces (`sbm/surrogate.py:14--21`):

- `classify_lambda(tree, oracle, lam, domain)` marks a cut cell as *retained* iff its domain-*outside* volume fraction $\leq \lambda$. So $\lambda = 0$ keeps only fully-interior cells (surrogate strictly inside Ω ; if the body is cell-aligned the surrogate coincides with Γ , $d = 0$); $\lambda = 0.5$ is the flow-past value.
- `extract_surrogate(tree)` returns the surrogate faces: faces of retained cells with no retained neighbour across them and not on the outer domain boundary (whole-face invariant).

6.2 The Taylor shift $S = N + \nabla N \cdot d$

Let d be the vector from a surrogate Gauss point to its closest point on Γ . The shifted trial/test operator is a first-order Taylor extrapolation from the surrogate to the true boundary:

$$S N_a = N_a + (\nabla N_a) \cdot d. \quad (6.1)$$

The Dirichlet data is imposed at the mapped point $y = x_q + d$, and the shifted trial value is $Su = u + (\nabla u) \cdot d$. When $d = 0$ (cell-aligned body), $S N_a = N_a$ and SBM is exactly standard Nitsche — the $d = 0$ case of Ch. 5.

Key idea. SBM is Nitsche with the shape functions Taylor-extrapolated to the true boundary. No re-meshing, no cut-cell quadrature: the shift is fixture *data* (d , *corr*), not solver code. The same predictor/PPE machinery runs unchanged.

6.3 Surrogate normal, distance, area correction

`GeometryData.evaluate` (`sbm/surrogate.py:222--253`) produces the frozen FP64 per-epoch geometry cache at surrogate Gauss points:

- d — the shift vector (surrogate GP $\rightarrow$ closest point on Γ), from the oracle’s `distance_vector`.
- n — the *true-boundary* outward normal $n = \pm \nabla \psi / |\nabla \psi|$, always pointing *out of* the computational domain Ω .
- **corr** — the area correction $\text{corr} = \tilde{n} \cdot n$, where $\tilde{n}$ is the surrogate face’s grid-aligned unit normal. It converts a surrogate-face measure to the true boundary measure ($\text{corr} = 1$ when $\tilde{\Gamma} = \Gamma$).

6.4 Force / drag on the immersed body

The drag reads the traction on the surrogate faces, area-corrected to the true boundary. `surrogate_traction(dm, sf, geo, x_all, nu, ndof)` (`sbm/vector.py:562--605`) computes

$$F = \oint_{\tilde{\Gamma}} (p n - \nu (\nabla u)^\top n) \text{corr} d\tilde{\Gamma}, \quad (6.2)$$

with the crucial **orientation contract**: `geo.n` is domain-outward (= *into* the obstacle for exterior flow), so the obstacle-outward normal is $\hat{n} = -\text{geo.n}$ and the integrand is $+p \text{geo.n} - \nu (\nabla u)^\top \text{geo.n}$. At $d = 0$ this reduces to the raw body-fitted traction.

6.5 Verification: a genuine sub-cell shift

The same square as Ch. 5, but its center is offset by a sub-cell fraction (`offset = 0.05`), so the grid-aligned surrogate no longer coincides with the true Box face: $0 < d_{\max} < h$. Now the Taylor $(\nabla N_a) \cdot d$ term *and* the area correction `corr` do real work. The stepper setup is *identical* to the weak path of Ch. 5 — the shift is fixture data. The monolithic oracle carries the *same* shifted geometry, so it is the same-mesh-with-shift oracle.

At level 4, $\text{Re} = 40$, `offset = 0.05`:

$$d_{\max} = \text{---} \quad (0 < d_{\max} < h = 0.0625, \quad d_{\max}/h = \text{---})$$

	projection	monolithic (true shift)	rel-diff
C_d	+-	+-	-

The consistent-projection split *with the genuine SBM shift active* matches the same-mesh-with-shift monolithic to - in drag — **projection + SBM works end-to-end in 2-D**.

Anti-vacuity (the shift is load-bearing). Run with `--zero-shift` to set `geo.d = 0`, `geo.corr = 1` while keeping everything else fixed; the match against the TRUE shifted oracle breaks — proof the Taylor term and area correction are doing real work.

Run it. `python run.py` in `04_shifted_boundary/` marches the shifted projection vs the shifted monolithic. `--zero-shift` demonstrates the anti-vacuity break; `--offset 0` returns to the body-fitted $d = 0$ case.

Q6.5.1 A

t offset = 0.05, $d_{\max}/h = 0.80$: the shift is nearly a full cell. What happens to the Taylor approximation as offset $\rightarrow h$? Why must the offset stay $< h$?

Q6.5.2 O

nly a handful of surrogate Gauss points are area-corrected (corr $\neq 1$). Which ones, geometrically, and why not all of them?

Chapter 7

Three dimensions — the SBM sphere

Module: 05_three_dimensions/. The end-to-end SBM-NS engine in 3-D; the shift $d \neq 0$ is genuinely active; the cuDSS/AMGX solver path; an honest verdict on the two engines.

7.1 Geometry

An immersed Sphere(center=(0.35,0.5,0.5), R=0.12) carved from a unit-box octree channel ($\lambda = 0.5$), ndof = 4, strong inflow ($x = 0$) + lateral walls (y, z), free outflow ($x = 1$), weak no-slip on the sphere. The sphere is not grid-aligned, so the SBM shift $d \neq 0$ is genuinely active in 3-D. At level 4: $n_{\text{free}} = --$, $-$ surrogate faces, $D/h = --$, $d_{\text{max}} = --$.

7.2 The honest 3-D verdict

The two engines part ways in 3-D, and the course is explicit about it (documented in tests/baselines/p2r0_task10_sphere.json):

MONOLITHIC 3-D SBM-NS — the working drag path. Stable and physical: it recovers from the startup transient to a positive steady drag,

$$C_d = + - - \quad (\text{Re} = 100, \alpha = 10, \text{level } 4),$$

reproducing the M1b sphere lock. This is the hero result for 3-D drag. It scales via GPU direct (solver="cudss") or block-preconditioned FGMRES past the host-splu wall ($\sim$ level 5 / 143k DOF).

The composition is de-risked (finite, axisymmetric $|C_{\text{lat}}| \ll |C_d|$, BDF2 engages, PPE-space weak divergence machine-zero), *but* its long-time drag transient is unstable at feasible 3-D mesh (the coupled iteration's C_d diverges monotonically while staying finite — a documented R2 item). So in 3-D we validate the pipeline invariants and take drag from the monolithic.

PROJECTION + volumetric SBM (LeraySBMStepper de-risked but not yet a drag path.

Key idea. The confined unit-box domain inflates C_d above the unconfined literature value; *both* engines see the same confinement, so the bar is faithfulness (and, in 3-D, the monolithic's physical steady state), not the literature number.

7.3 Solver path

Host `splu` is used for the small level-4 case here ($\sim$ seconds per step). To scale up, build the fixture on a CUDA device and route the monolithic solve through `solver="cudss"` (GPU direct) or `solver="blockamgx"` (block-preconditioned FGMRES: a Cahouet–Chabard Schur complement + AMG-on- F), both wired in the sphere fixture’s `monolithic_cd` (`p2r0_task10_sphere_derisk.py:179--281`). The *projection* engine’s SPD PPE is the AMG-friendly one; that is the subject of Ch. 8.

7.4 Verification

quantity	value (level 4, Re = 100)
n_{free}	–
sphere surrogate faces	–
D/h	–
SBM d_{max}	– (> 0 : genuine 3-D shift)
MONOLITHIC steady C_d	+ – – (positive, physical)
$ C_{\text{lat}} / C_d $ (pipeline leg)	– (axisymmetric)

Run it. `python run.py` in `05_three_dimensions/` marches the monolithic drag path (the hero number). Add `--pipeline` to additionally check the projection composition invariants over a short window (finite, BDF2 engaged, axisymmetric traction) — NOT the unstable long march.

Q7.4.1 A

t level 5 the host-splu factorization runs out of time/memory. Estimate the DOF count and argue why `cudss` (GPU direct) buys you one more level but not many. What does that imply for the projection path?

Q7.4.2 T

he projection composition is finite and axisymmetric yet its drag transient diverges. Is “finite” the same as “stable”? What diagnostic distinguishes them here?

Chapter 8

Toward 100M DOF (planned)

PLANNED — not yet written. This chapter and its module (`06_toward_100M_dof/`) are a placeholder. The outline below and the source pointers are what a future session will build from; see `HANDOFF.md` at the course root for the full, actionable instructions.

Module: `06_toward_100M_dof/` (stub). The pressure-projection engine's reason to exist: its SPD pressure-Poisson admits algebraic multigrid, so it — unlike the indefinite saddle — reaches hundred-million-DOF meshes.

What this chapter will teach

- **The scaling verdict.** The cuDSS direct solver *cannot* factorize the film hero (15.15M DOF, 1.6B nnz) on a GH200 — an HBM-saturation capacity wall, identical in fp64 and fp32. So the 100M-DOF pathway is **iterative, not direct**, which is exactly why the projection engine (SPD-PPE → AMG/AMGX) is the load-bearing choice.
- **The AMGX PPE study (task W1).** The Leray stepper's SPD PPE driven through AMGX (PCG + classical-AMG V-cycle) on 3-D uniform meshes: per-solve time, AMG iteration count, residual, memory, and the single-GPU DOF ceiling. Textbook AMG scaling; 100M fits in roughly 33 GB/GPU. Parity check vs `splu` at small levels.
- **Device K_p assembly (the mesh-build wall).** The A100/GH200 forensics found a ~ 61 s/step host-side floor (assembly / scatter / GP fields) common to all three GPU generations — a silent Amdahl ceiling. The chapter measures it, then shows the device-assembled K_p path that removes it, and the *one-time* mesh-build cost that remains.
- **The L8→GH200 path.** The PPE scalar DOF map for a 3-D uniform octree at level L is $n = (2^L + 1)^3$: L7 ~ 2.1 M, L8 ~ 16.6 M, L9 ~ 133 M. 100M sits between L8 and L9. The study reveals where the single-GPU ceiling actually bites (the Warp 2^{31} array-shape limit vs host COO→CSR assembly) and the deployment tier (single big node: GH200/NVL4) it targets.

Source material (for the author of this chapter)

- `tests/ppe_amgx_scaling.py` — the W1 weak/strong-scaling harness (the real brick this module drives; run on the box in a proj lane with `LD_LIBRARY_PATH` set for AMGX).

- The scaling-pathway program rule and the “iterative-not-direct” conclusion: `docs/dev/production-code-conventions.md` (scaling-pathway section) and the SP-1/P2 milestone notes.
- `docs/dev/2026-07-23-projection-ladder-FINAL-synthesis.md` — the projection engine’s validation state.
- The stage-residency + 100M-DOF-budget table format (bytes/dof, nnz/dof, index width, multi-GPU comms) every scaling chapter must carry.

Target numbers (to be measured and locked)

Per-level PPE solve time and AMG iteration count at L5–L8; the `amgx-vs-splu` parity residual at L4–L5; the measured bytes/dof and the projected 100M-DOF single-GPU memory (~ 33 GB target).

Chapter 9

Multi-GPU (planned)

PLANNED — not yet written. This chapter and its module (`07_multi_gpu/`) are a placeholder. The outline below is what a future session will build from; see `HANDOFF.md` for the full instructions.

Module: `07_multi_gpu/` (stub). Beyond a single GPU: the distributed solve for problems whose DOF count exceeds one card's memory — the multi-node deployment tier.

What this chapter will teach

- **When one GPU is not enough.** Chapter 8 reaches $\sim 100\text{M}$ DOF on a single big node (GH200/NVL4). The next tier — the film hero and finer meshes — needs a *distributed* solve. This chapter is the roadmap (Track S / the P2 milestone).
- **Device-FGMRES + distributed AMG.** The mandated iterative path (cuDSS cannot factorize the hero) makes a device-resident FGMRES with a distributed AMG/AMGX preconditioner the load-bearing kernel. The multi-GPU comms pattern (halo exchange for the SpMV, the coarse-grid gather) is the design surface.
- **The partitioning + comms budget.** The 100M-DOF budget line (bytes/dof, nnz/dof, index width per the mixed-width rule, comms volume) extended to N GPUs; the deployment-tier declaration (Horizon gb-large / AWS on-demand clusters).

Source material (for the author of this chapter)

- The roadmap reconciliation (Track P / Track S) and the P2 NSHT-SBM-Shell hero spec (the multi-node hero this chapter serves).
- The device-FGMRES design note (mandatory for the film hero per the scaling conclusion).
- Chapter 8's single-GPU AMGX study as the baseline the multi-GPU path must weak-scale from.
- The remote-deploy toolkit (`scripts/remote/`) for the gpubox/Nova multi-GPU wiring.

Status

Future. This is a roadmap chapter: the single-GPU AMGX result (Ch. 8) must land first — it defines the per-GPU baseline the distributed solve weak-scales from.

Chapter 10

Differentiable structure (planned)

PLANNED — not yet written. This chapter and its module (`08_differentiable_structure/`) are a placeholder. The outline below is what a future session will build from; see `HANDOFF.md` for the full instructions.

Module: `08_differentiable_structure/` (stub). The NS-SBM engine as an instrument for design: gradients through the solve, and shape optimization of the immersed body.

What this chapter will teach

- **The adjoint of the NS-SBM engine.** A gradient of a flow QoI (drag, a wake observable) with respect to a control — the SBM shift / the body shape, the Reynolds number, a boundary trace. The steady adjoint is a *single transposed solve* (no trajectory), which is exactly what makes it tractable at 100M DOF.
- **Three-way-verified gradients.** Following the sibling differentiable track’s discipline: custom adjoint == autograd twin == finite difference, on the *same* operator the forward stepper marches. The course’s standing bar is that a gradient is not trusted until all three agree.
- **Shape optimization on the shifted boundary.** Because the SBM shift is fixture *data* (*d*, corr; Ch. 6), differentiating with respect to the body geometry flows through the distance field — a natural, mesh-free shape-design surface.

Source material (for the author of this chapter)

- The SBM adjoint / shape bricks already in the tree: `src/diffsim/sbm/ns_adjoint.py`, `leray_adjoint.py`, `ns_shape.py`, `transient_adjoint.py`, `adjoint.py`.
- The SP-1 R1 XDD adjoint work (MERGED fbfeed5): the three-way-verification pattern (adjoint vs torch-twin vs FD), the unified `Control` interface, the exposed sensitivity rows — the template this chapter mirrors for the NS-SBM operator.
- The P2 milestone’s “steady adjoint = 1 transposed solve, tractable at 100M” argument (deliverable C, the maize steady-differentiable design engine).
- The sibling `orgelmorph-course/differentiable/` track for the module template of a gradient chapter (core module, `run.py` printing the three-way agreement, `EXPECTED.md`).

Status

Future. Ties into the E-differentiable track and the SP-1 R1 adjoint work; best built after the forward engine chapters (00–05) and the scaling chapter (06) are the students’ shared baseline.

Bibliography

- [1] N. M. Atallah, C. Canuto, and G. Scovazzi. The second-generation shifted boundary method and its application to Stokes flows. *Computer Methods in Applied Mechanics and Engineering*, 372:113341, 2020.
- [2] Y. Bazilevs, V. M. Calo, J. A. Cottrell, T. J. R. Hughes, A. Reali, and G. Scovazzi. Variational multiscale residual-based turbulence modeling for large eddy simulation of incompressible flows. *Computer Methods in Applied Mechanics and Engineering*, 197:173–201, 2007.
- [3] Y. Bazilevs, J. R. Gohean, T. J. R. Hughes, R. D. Moser, and Y. Zhang. Patient-specific isogeometric fluid–structure interaction analysis of thoracic aortic blood flow due to implantation of the Jarvik 2000 left ventricular assist device. *Computer Methods in Applied Mechanics and Engineering*, 198:3534–3550, 2009.
- [4] A. N. Brooks and T. J. R. Hughes. Streamline upwind/Petrov–Galerkin formulations for convection dominated flows with particular emphasis on the incompressible Navier–Stokes equations. *Computer Methods in Applied Mechanics and Engineering*, 32:199–259, 1982.
- [5] A. J. Chorin. Numerical solution of the Navier–Stokes equations. *Mathematics of Computation*, 22:745–762, 1968.
- [6] DiffSim project. The `consistent_projection` fix (#1–#6) — design. Technical report, 2026. docs/dev/specs/2026-07-23-consistent-projection-fix-design.md.
- [7] DiffSim project. Projection validation ladder — final synthesis (rungs 0–c). Technical report, 2026. docs/dev/2026-07-23-projection-ladder-FINAL-synthesis.md.
- [8] M. Esmaily-Moghadam, Y. Bazilevs, T.-Y. Hsia, I. E. Vignon-Clementel, and A. L. Marsden. A comparison of outlet boundary treatments for prevention of backflow divergence with relevance to blood flow simulations. *Computational Mechanics*, 48:277–291, 2011.
- [9] U. Ghia, K. N. Ghia, and C. T. Shin. High-Re solutions for incompressible flow using the Navier–Stokes equations and a multigrid method. *Journal of Computational Physics*, 48(3): 387–411, 1982.
- [10] T. J. R. Hughes, G. R. Feijóo, L. Mazzei, and J.-B. Quincy. The variational multiscale method—a paradigm for computational mechanics. *Computer Methods in Applied Mechanics and Engineering*, 166:3–24, 1998.
- [11] A. Main and G. Scovazzi. The shifted boundary method for embedded domain computations. Part I: Poisson and Stokes problems. *Journal of Computational Physics*, 372:972–995, 2018.

- [12] J. Nitsche. Über ein variationsprinzip zur lösung von Dirichlet-problemen bei verwendung von teilräumen, die keinen randbedingungen unterworfen sind. *Abhandlungen aus dem Mathematischen Seminar der Universität Hamburg*, 36:9–15, 1971.
- [13] D. R. Q. Pacheco, R. Schussnig, O. Steinbach, and T.-P. Fries. A global Newton-type scheme and boundary-vorticity outflow treatment for stabilized incompressible flow solvers. *International Journal for Numerical Methods in Engineering*, 2021. arXiv:2411.02100.
- [14] T. E. Tezduyar. Stabilized finite element formulations for incompressible flow computations. *Advances in Applied Mechanics*, 28:1–44, 1992.
- [15] L. J. P. Timmermans, P. D. Minev, and F. N. Van De Vosse. An approximate projection scheme for incompressible flow using spectral elements. *International Journal for Numerical Methods in Fluids*, 22:673–688, 1996.